

Precedence in Java Expression Evaluation

- SUB-EXPRESSIONS ARE EVALUATED FROM LEFT TO RIGHT (WHEN THERE IS A CHOICE)**

In the expression $A() + B() * C(D(), E())$, the subexpressions are evaluated in the order $A()$, $B()$, $D()$, $E()$, and $C()$. Although, $C()$ appears to the left of both $D()$ and $E()$, we need the results of both $D()$ and $E()$ to evaluate $C()$. Also, even though $*$ has a higher precedence than $+$, $A()$ is the left sub-expression of the addition and is evaluated first, followed by the left sub-expression of $*$, which is $B()$.

Note that it is considered poor style to write code that relies upon this behavior!!!

Also, $(x + x) + ++x = 16$ given $x = 5$ because the precedence rules guarantee that both the left-most addition and the increment operator will occur before the right-hand addition, but doesn't cause the increment to happen first.

$(X_{(1)} +_{(3)} X_{(2)}) +_{(6)} ++_{(5)} X_{(4)}$
 $(5 + 5) + 6 \rightarrow 16$

- Operators in the same numbered row in the chart have equal precedence and are evaluated according to the association listed in the Order of Evaluation column. (Left to Right or Right to Left)
- Underlined operators have side-effects. They can be used as statements if a semi-colon is appended.
Note that care should be exercised to make code clear, not clever. Use at most one side-effect per expression.

Just remember this:

- Unary operators**
- Arithmetic operators**
numbers $\rightarrow$ number
- Relational operators**
numbers $\rightarrow$ boolean
- Boolean operators**
booleans $\rightarrow$ boolean
- Assignment operators**

Use () for fine-grain control!

Priority	Operator	Operation	Signature	Order of Evaluation	Notes
0	ALL SUBEXPRESSIONS ARE EVALUATED FROM LEFT TO RIGHT (WHEN POSSIBLE)!!!			Left to Right	
1	<code>array[index]</code>	Array access	array x int $\rightarrow$ element	Left to Right	In <code>new Class().func()</code> the constructor is called and then the method. In <code>new Outer.Inner()</code> the outer class is identified first and then the inner constructor is called. The order is reversed due to the rules of Java syntax, not operator precedence and associativity rules.
	<code>mthd(args)</code>	Method call	method x args $\rightarrow$ result?		
	<code>construct(args)</code>	<u>Constructor (Object creation)</u>	Class x args $\rightarrow$ object		
	<code>.</code>	Member access	object x member $\rightarrow$ member		
2	<code>exp++</code>	<u>Postfix increment</u>	variable $\rightarrow$ value	No associative	These both return the original value of the variable.
	<code>exp--</code>	<u>Postfix decrement</u>			
3	<code>++exp</code>	<u>Prefix increment</u>	variable $\rightarrow$ value	Right to Left	These both return the new value of the variable.
	<code>--exp</code>	<u>Prefix decrement</u>			
	<code>+ -</code>	Unary plus, minus	number $\rightarrow$ number		<code>-a % b = -(a % b)</code> , but <code>a % -b = a % b</code> eg: <code>-7 % 4 = -3</code> , but <code>7 % -4 = 3</code>
	<code>~</code>	Bitwise NOT	integer $\rightarrow$ integer		<code>~</code> flips each bit of a number
	<code>!</code>	Logical NOT	boolean $\rightarrow$ boolean		
4	<code>new</code>	<u>Object creation</u>	class(args) $\rightarrow$ object	Right to Left	See note to level 1
	<code>(type)</code>	Type cast	(type) ref / value $\rightarrow$ type		

5	* / %	Multiplication, division, remainder	number x number → number	Left to Right	WARNING: Integer types will overflow without giving an error.
6	+ -	Addition, subtraction	number x number → number	Left to Right	same as for * / %
	+	String concatenation	string x any → string		Add "" to force concatenation
7	<<	Signed bit shift to the left	integer x integer → integer	Left to Right	
	>>	Signed bit shift to the right	integer x integer → integer		
	>>>	Unsigned bit shift to the right	integer x integer → integer		
8	< <=	Less than, less than or equal to	number x number → boolean	Left to Right	
	> >=	Greater than, greater than or equal to	number x number → boolean		
	instanceof	Reference test	ref x type → boolean		null instanceof T always returns false
9	==	Equal to (<i>identical value</i>)	primitive x primitive → boolean	Left to Right	
	!=	Not equal to (<i>identical value</i>)	primitive x primitive → boolean		
	==	Equal to (<i>same object</i>)	ref x ref → boolean	Left to Right	Use equals () to check for structural equality
	!=	Not equal to (<i>different object</i>)	ref x ref → boolean		
10	&	Bitwise AND	integer x integer → integer	Left to Right	
	&	Full evaluation Logical AND	boolean x boolean → boolean		
11	^	Bitwise XOR	number x number → number	Left to Right	Note that the ^ symbol is <u>not</u> for exponents! Use Math.pow() for that.
	^	Full evaluation Logical XOR	boolean x boolean → boolean		
12	 	Bitwise OR	integer x integer → integer	Left to Right	
	 	Full evaluation Logical OR	boolean x boolean → boolean		
13	&&	Short-circuited Logical AND	boolean x boolean → boolean	Left to Right	
14	 	Short-circuited Logical OR	boolean x boolean → boolean	Left to Right	
15	? :	Ternary Conditional (if-then-else)	boolean x any x any → any	Right to Left	minVal = (a<b) ? a : b; is the same as: if(a<b) minVal = a; else minVal = b;
16	=	<u>Assignment</u>	variable x any → any	Right to Left	Floating-points converted to integers are truncated, not rounded.
	*= /= += -= %= <<= >>= >>>= &= ^= =	<u>Combined assignment</u> (operation and assignment)	variable x any → any		Note that int x=5; x += 2.0; works bc x += 2.0 is equivalent to: x = (int)(x + 2.0);
17	,	comma operator	any x any → any	Left to Right	The comma operator is rarely used. It evaluates each expression in order, then returns the value of the last.